

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

MODUL XIV

GRAPH



Disusun Oleh :

Muhammad Fathammubina

NIM : 103112430188

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

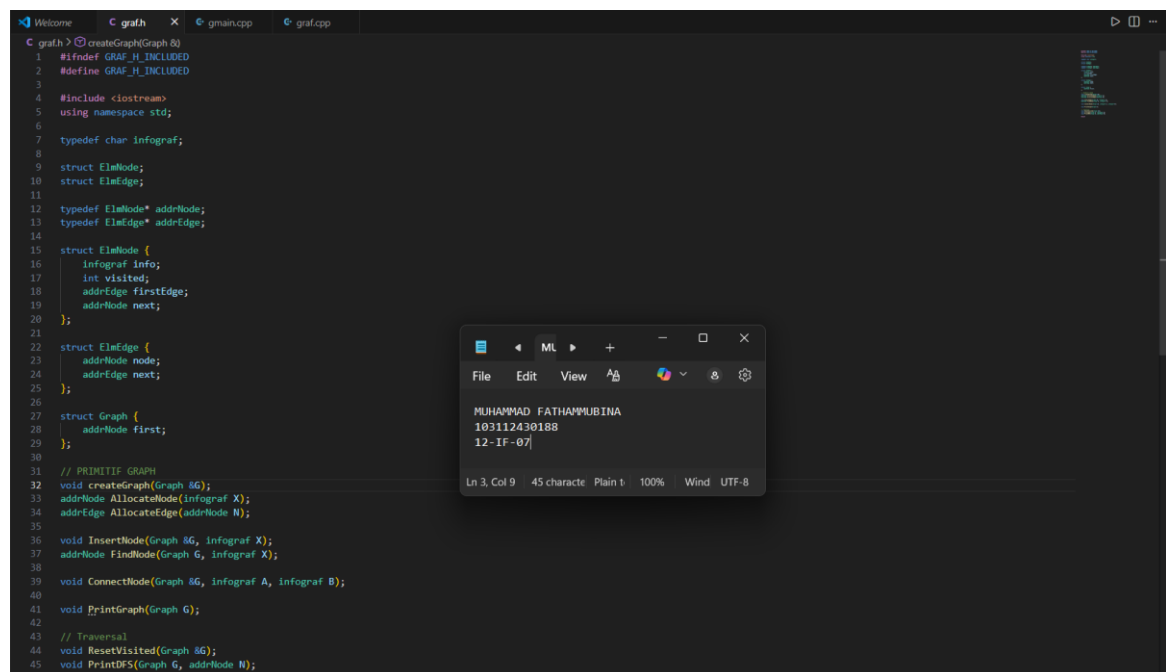
A. Dasar Teori

Graph adalah struktur data yang dipakai untuk menggambarkan hubungan antar elemen. Graph tersusun dari node/vertex sebagai penyimpan data, dan edge sebagai penghubung antar node. Berdasarkan arahnya, graph dibagi menjadi directed (punya arah) dan undirected (tanpa arah, dua arah). Salah satu cara merepresentasikan graph adalah dengan adjacency list berbasis linked list, di mana setiap node menyimpan daftar edge ke node tetangganya, serta memiliki variabel visited untuk menandai apakah node sudah dikunjungi saat traversal. Graph dapat ditelusuri dengan algoritma seperti DFS yang menjelajah sedalam mungkin ke tetangga sebelum kembali, dan BFS yang menjelajah per level dimulai dari node awal menggunakan antrean. Graph banyak digunakan pada kasus seperti jaringan komputer, peta rute, dan relasi antar objek.

B. Guided

Guided 1

graf.h



```
1 #ifndef GRAF_H_INCLUDED
2 #define GRAF_H_INCLUDED
3
4 #include <iostream>
5 using namespace std;
6
7 typedef char infograf;
8
9 struct ElmNode;
10 struct ElmEdge;
11
12 typedef ElmNode* addrNode;
13 typedef ElmEdge* addrEdge;
14
15 struct ElmNode {
16     infograf info;
17     int visited;
18     addrEdge firstEdge;
19     addrNode next;
20 };
21
22 struct ElmEdge {
23     addrNode node;
24     addrEdge next;
25 };
26
27 struct Graph {
28     addrNode first;
29 };
30
31 // PRIMITIF GRAPH
32 void createGraph(Graph &G);
33 addrNode AllocateNode(infograf X);
34 addrEdge AllocateEdge(addrNode N);
35
36 void InsertNode(Graph &G, infograf X);
37 addrNode FindNode(Graph G, infograf X);
38
39 void ConnectNode(Graph &G, infograf A, infograf B);
40
41 void PrintGraph(Graph G);
42
43 // Traversal
44 void ResetVisited(Graph &G);
45 void PrintDFS(Graph G, addrNode B);
```

```
#ifndef GRAF_H_INCLUDED
#define GRAF_H_INCLUDED

#include <iostream>
using namespace std;

typedef char infograf;

struct ElmNode;
struct ElmEdge;

typedef ElmNode* addrNode;
```

```

typedef ElmEdge* addrEdge;

struct ElmNode {
    infograf info;
    int visited;
    addrEdge firstEdge;
    addrNode next;
};

struct ElmEdge {
    addrNode node;
    addrEdge next;
};

struct Graph {
    addrNode first;
};

// PRIMITIF GRAPH
void createGraph(Graph &G);
addrNode AllocateNode(infograf X);
addrEdge AllocateEdge(addrNode N);

void InsertNode(Graph &G, infograf X);
addrNode FindNode(Graph G, infograf X);

void ConnectNode(Graph &G, infograf A, infograf B);

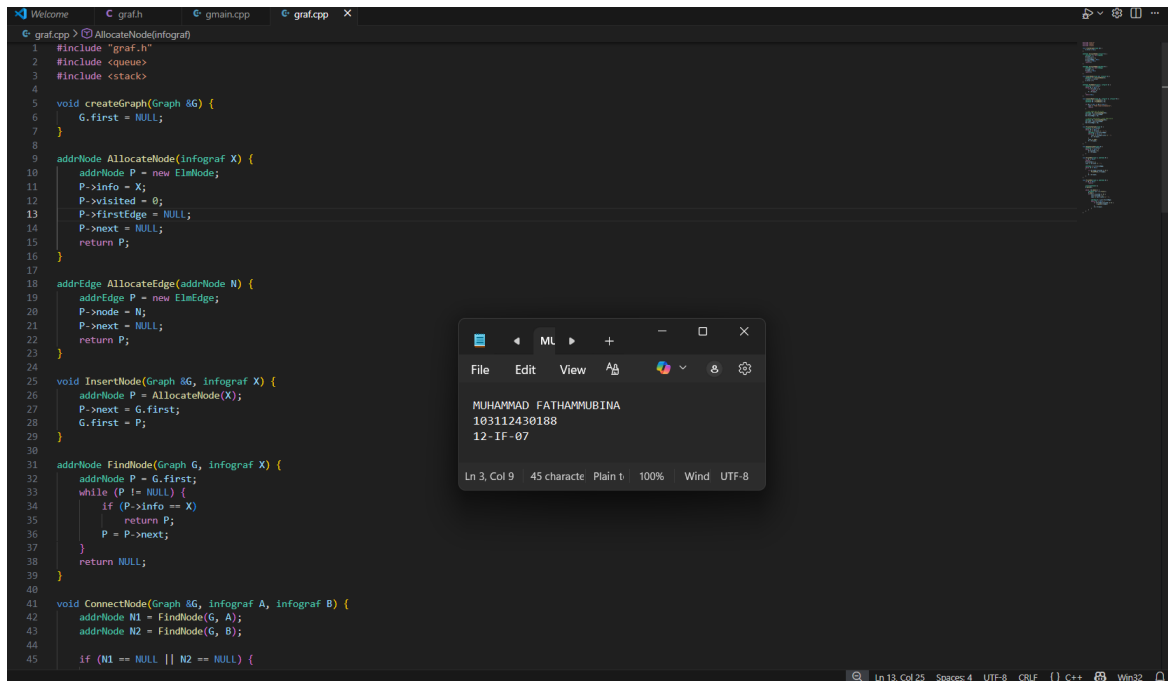
void PrintGraph(Graph G);

// Traversal
void ResetVisited(Graph &G);
void PrintDFS(Graph G, addrNode N);
void PrintBFS(Graph G, addrNode N);

#endif

```

graf.cpp



```
1 #include "graf.h"
2 #include <queue>
3 #include <stack>
4
5 void createGraph(Graph &G) {
6     G.first = NULL;
7 }
8
9 addrNode AllocateNode(infograf X) {
10     addrNode P = new ElmNode;
11     P->info = X;
12     P->visited = 0;
13     P->firstEdge = NULL;
14     P->next = NULL;
15     return P;
16 }
17
18 addrEdge AllocateEdge(addrNode N) {
19     addrEdge P = new ElmEdge;
20     P->node = N;
21     P->next = NULL;
22     return P;
23 }
24
25 void InsertNode(Graph &G, infograf X) {
26     addrNode P = AllocateNode(X);
27     P->next = G.first;
28     G.first = P;
29 }
30
31 addrNode FindNode(Graph G, infograf X) {
32     addrNode P = G.first;
33     while (P != NULL) {
34         if (P->info == X)
35             return P;
36         P = P->next;
37     }
38     return NULL;
39 }
40
41 void ConnectNode(Graph &G, infograf A, infograf B) {
42     addrNode N1 = FindNode(G, A);
43     addrNode N2 = FindNode(G, B);
44
45     if (N1 == NULL || N2 == NULL) {
```

```
#include "graf.h"
#include <queue>
#include <stack>

void createGraph(Graph &G) {
    G.first = NULL;
}

addrNode AllocateNode(infograf X) {
    addrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
    P->firstEdge = NULL;
    P->next = NULL;
    return P;
}

addrEdge AllocateEdge(addrNode N) {
    addrEdge P = new ElmEdge;
    P->node = N;
    P->next = NULL;
    return P;
}

void InsertNode(Graph &G, infograf X) {
    addrNode P = AllocateNode(X);
    P->next = G.first;
    G.first = P;
}
```

```

addrNode FindNode(Graph G, infograf X) {
    addrNode P = G.first;
    while (P != NULL) {
        if (P->info == X)
            return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(Graph &G, infograf A, infograf B) {
    addrNode N1 = FindNode(G, A);
    addrNode N2 = FindNode(G, B);

    if (N1 == NULL || N2 == NULL) {
        cout << "Node tidak ditemukan\n";
        return;
    }

    // Buat Edge dari N1 ke N2
    addrEdge E1 = AllocateEdge(N2);
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;

    // Karena undirected -> buat Edge balik
    addrEdge E2 = AllocateEdge(N1);
    E2->next = N2->firstEdge;
    N2->firstEdge = E2;
}

void PrintInfoGraph(Graph G) {
    addrNode P = G.first;
    while (P != NULL) {
        cout << P->info << " -> ";
        addrEdge E = P->firstEdge;
        while (E != NULL) {
            cout << E->node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

void ResetVisited(Graph &G) {
    addrNode P = G.first;
    while (P != NULL) {

```

```

        P->visited = 0;
        P = P->next;
    }
}

void PrintDFS(Graph G, addrNode N) {
    if (N == NULL)
        return;
    N->visited = 1;
    cout << N->info << " ";

    addrEdge E = N->firstEdge;
    while (E != NULL)
    {
        if (E->node->visited == 0) {
            PrintDFS(G, E->node);
        }
        E = E->next;
    }
}

void PrintBFS(Graph G, addrNode N) {
    if (N == NULL)
        return;

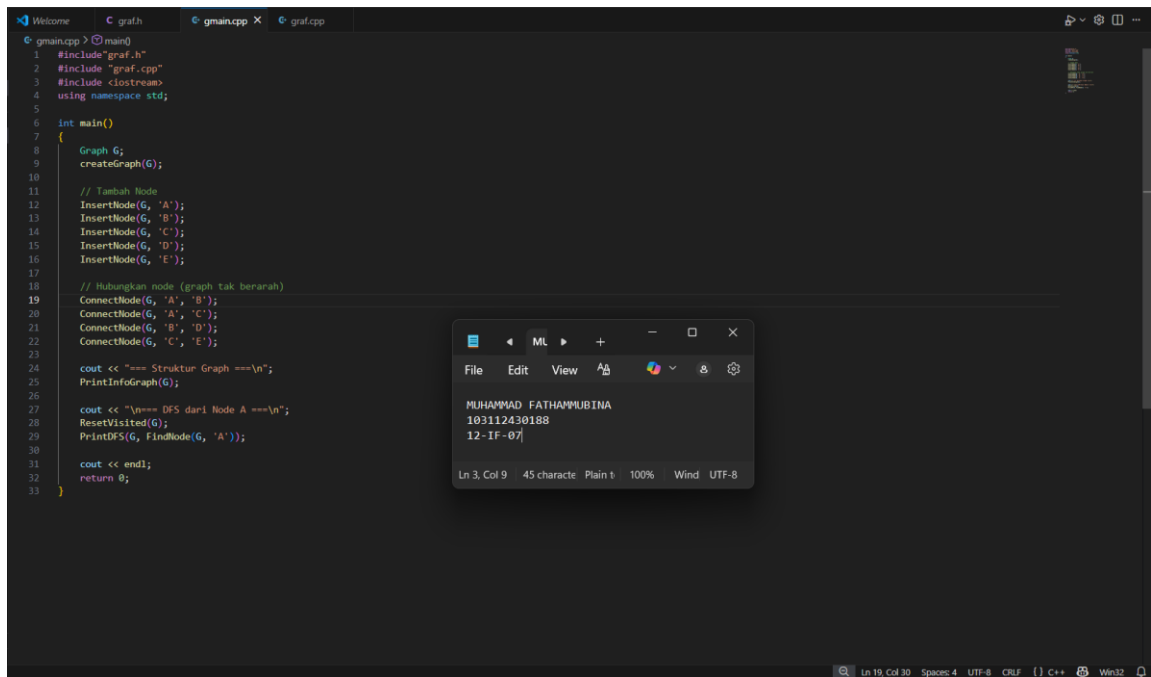
    queue<addrNode> Q;
    Q.push(N);

    while (!Q.empty()) {
        addrNode curr = Q.front();
        Q.pop();
        if (curr->visited == 0) {
            curr->visited = 1;
            cout << curr->info << " ";

            addrEdge E = curr->firstEdge;
            while (E != NULL) {
                if (E->node->visited == 0) {
                    Q.push(E->node);
                }
                E = E->next;
            }
        }
    }
}

```

gmain.cpp



```
1 #include "graf.h"
2 #include "graf.cpp"
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     Graph G;
9     createGraph(G);
10
11     // Tambah Node
12     InsertNode(G, 'A');
13     InsertNode(G, 'B');
14     InsertNode(G, 'C');
15     InsertNode(G, 'D');
16     InsertNode(G, 'E');
17
18     // Hubungkan node (graph tak berarah)
19     ConnectNode(G, 'A', 'B');
20     ConnectNode(G, 'A', 'C');
21     ConnectNode(G, 'B', 'D');
22     ConnectNode(G, 'C', 'E');
23
24     cout << "=== Struktur Graph ===\n";
25     PrintInfoGraph(G);
26
27     cout << "\n=== DFS dari Node A ===\n";
28     ResetVisited(G);
29     PrintDFS(G, FindNode(G, 'A'));
30
31     cout << endl;
32     return 0;
33 }
```

MUHAMMAD FATHAMMUBINA
103112430188
12-IF-07

Ln 3, Col 9 45 character Plain text 100% Wind UTF-8

```
#include "graf.h"
#include "graf.cpp"
#include <iostream>
using namespace std;

int main()
{
    Graph G;
    createGraph(G);

    // Tambah Node
    InsertNode(G, 'A');
    InsertNode(G, 'B');
    InsertNode(G, 'C');
    InsertNode(G, 'D');
    InsertNode(G, 'E');

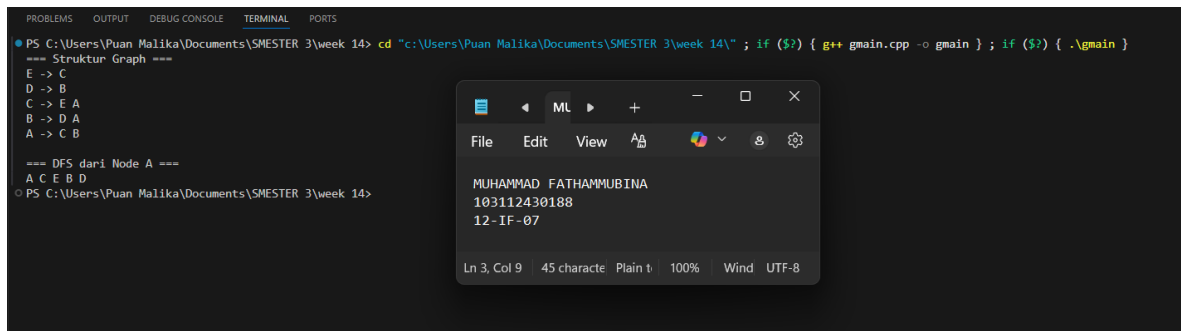
    // Hubungkan node (graph tak berarah)
    ConnectNode(G, 'A', 'B');
    ConnectNode(G, 'A', 'C');
    ConnectNode(G, 'B', 'D');
    ConnectNode(G, 'C', 'E');

    cout << "=== Struktur Graph ===\n";
    PrintInfoGraph(G);

    cout << "\n=== DFS dari Node A ===\n";
    ResetVisited(G);
    PrintDFS(G, FindNode(G, 'A'));
```

```
    cout << endl;
    return 0;
}
```

Screenshots Output



```
PS C:\Users\Puan Malika\Documents\SEMESTER 3\week 14> cd "c:\Users\Puan Malika\Documents\SEMESTER 3\week 14\" ; if ($?) { g++ gmain.cpp -o gmain } ; if ($?) { .\gmain }
=== Struktur Graph ===
E -> C
D -> B
C -> E A
B -> D A
A -> C B

=== DFS dari Node A ===
A C E B D
PS C:\Users\Puan Malika\Documents\SEMESTER 3\week 14>
```

Deskripsi:

Program di atas adalah implementasi ADT Graph (graf tak berarah) memakai linked list. Graf menyimpan data berupa huruf (char) sebagai node, dan hubungan antar node disimpan sebagai edge dalam daftar bertaut. Setiap node punya penanda visited untuk traversal, serta daftar edge yang mengarah ke node tetangga. Program menyediakan operasi dasar seperti membuat graf (createGraph), menambah node (InsertNode), mencari node (FindNode), dan menghubungkan dua node (ConnectNode) secara dua arah karena graf bersifat undirected. Ada juga fitur traversal yaitu DFS (rekursif) dan BFS (antrean) untuk menelusuri graf, serta fungsi ResetVisited agar node bisa dikunjungi ulang. Di main, graf dibuat lalu diisi node A–E, disambungkan sesuai pola tertentu, kemudian ditampilkan strukturnya dan dilakukan DFS mulai dari node A. Program ini berguna untuk merepresentasikan dan menjelajahi hubungan antar data secara dinamis.

Unguided 1

graph.h

```
C graph.h:~
1  #ifndef GRAPH_H_INCLUDED
2  #define GRAPH_H_INCLUDED
3
4  #include <iostream>
5  using namespace std;
6
7  typedef char infoGraph;
8
9  struct ElmNode;
10 struct ElmEdge;
11
12 typedef ElmNode* adrNode;
13 typedef ElmEdge* adrEdge;
14
15 struct ElmNode {
16     infoGraph info;
17     int visited;
18     adrEdge firstEdge;
19     adrNode next;
20 };
21
22 struct ElmEdge {
23     adrNode node;
24     adrEdge next;
25 };
26
27 struct Graph {
28     adrNode first;
29 };
30
31 // Primitif sesuai soal latihan 1
32 void CreateGraph(Graph &G);
33 void InsertNode(Graph &G, infoGraph X);
34 void ConnectNode(adrNode N1, adrNode N2);
35 void PrintInfoGraph(Graph G);
36
37 // Primitif tambahan untuk mendukung
38 adrNode AllocateNode(infoGraph X);
39 adrEdge AllocateEdge(adrNode N);
40 adrNode FindNode(Graph G, infoGraph X);
41 void ResetVisited(Graph &G);
42
43 // Soal 2 dan 3
44 void PrintDFS(Graph G, adrNode N);
45 void PrintBFS(Graph G, adrNode N);
```

```
#ifndef GRAPH_H_INCLUDED
#define GRAPH_H_INCLUDED

#include <iostream>
using namespace std;

typedef char infoGraph;

struct ElmNode;
struct ElmEdge;

typedef ElmNode* adrNode;
typedef ElmEdge* adrEdge;

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
};

struct ElmEdge {
    adrNode node;
    adrEdge next;
};
```

```

struct Graph {
    adrNode first;
};

// Primitif sesuai soal latihan 1
void CreateGraph(Graph &G);
void InsertNode(Graph &G, infoGraph X);
void ConnectNode(adNode N1, adrNode N2);
void PrintInfoGraph(Graph G);

// Primitif tambahan untuk mendukung
adrNode AllocateNode(infoGraph X);
adrEdge AllocateEdge(adNode N);
adrNode FindNode(Graph G, infoGraph X);
void ResetVisited(Graph &G);

// Soal 2 dan 3
void PrintDFS(Graph G, adrNode N);
void PrintBFS(Graph G, adrNode N);

#endif

```

graph.cpp

```

graph.cpp > PrintBFS(Graph, adrNode)
1  #include "graph.h"
2  #include <queue>
3
4  void CreateGraph(Graph &G) {
5      G.first = NULL;
6  }
7
8  adrNode AllocateNode(infoGraph X) {
9      adrNode P = new ElemNode;
10     P->info = X;
11     P->visited = 0;
12     P->firstEdge = NULL;
13     P->next = NULL;
14     return P;
15 }
16
17 adrEdge AllocateEdge(adNode N) {
18     adrEdge P = new ElemEdge;
19     P->nnode = N;
20     P->next = NULL;
21     return P;
22 }
23
24 void InsertNode(Graph &G, infoGraph X) {
25     adrNode P = AllocateNode(X);
26     P->next = G.first;
27     G.first = P;
28 }
29
30 adrNode FindNode(Graph G, infoGraph X) {
31     adrNode P = G.first;
32     while (P != NULL) {
33         if (P->info == X)
34             return P;
35         P = P->next;
36     }
37     return NULL;
38 }
39
40 void ConnectNode(adNode N1, adrNode N2) {
41     if (N1 == NULL || N2 == NULL) {
42         return;
43     }
44
45     // Edge dari N1 ke N2

```

```

File Edit View
MUHAMMAD FATHAMMUBINA
103112430188
12-1F-07
Ln 3, Col 9 45 character Plain t 100% Wind UTF-8

```

```

#include "graph.h"
#include <queue>

void CreateGraph(Graph &G) {
    G.first = NULL;

```

```

}

adrNode AllocateNode(infoGraph X) {
    adrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
    P->firstEdge = NULL;
    P->next = NULL;
    return P;
}

adrEdge AllocateEdge(adrNode N) {
    adrEdge P = new ElmEdge;
    P->node = N;
    P->next = NULL;
    return P;
}

void InsertNode(Graph &G, infoGraph X) {
    adrNode P = AllocateNode(X);
    P->next = G.first;
    G.first = P;
}

adrNode FindNode(Graph G, infoGraph X) {
    adrNode P = G.first;
    while (P != NULL) {
        if (P->info == X)
            return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(adrNode N1, adrNode N2) {
    if (N1 == NULL || N2 == NULL) {
        return;
    }

    // Edge dari N1 ke N2
    adrEdge E1 = AllocateEdge(N2);
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;

    // Edge dari N2 ke N1 (undirected)
    adrEdge E2 = AllocateEdge(N1);
    E2->next = N2->firstEdge;
    N2->firstEdge = E2;
}

```

```

}

void PrintInfoGraph(Graph G) {
    adrNode P = G.first;
    while (P != NULL) {
        cout << P->info << " -> ";
        adrEdge E = P->firstEdge;
        while (E != NULL) {
            cout << E->node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

void ResetVisited(Graph &G) {
    adrNode P = G.first;
    while (P != NULL) {
        P->visited = 0;
        P = P->next;
    }
}

// Soal 2: Prosedur PrintDFS
void PrintDFS(Graph G, adrNode N) {
    if (N == NULL)
        return;

    N->visited = 1;
    cout << N->info << " ";

    adrEdge E = N->firstEdge;
    while (E != NULL) {
        if (E->node->visited == 0) {
            PrintDFS(G, E->node);
        }
        E = E->next;
    }
}

// Soal 3: Prosedur PrintBFS
void PrintBFS(Graph G, adrNode N) {
    if (N == NULL)
        return;

    queue<adrNode> Q;
    Q.push(N);

```

```

while (!Q.empty()) {
    adrNode curr = Q.front();
    Q.pop();

    if (curr->visited == 0) {
        curr->visited = 1;
        cout << curr->info << " ";

        adrEdge E = curr->firstEdge;
        while (E != NULL) {
            if (E->node->visited == 0) {
                Q.push(E->node);
            }
            E = E->next;
        }
    }
}
}

```

graphmain.cpp

```

1 #include "graph.h"
2
3 int main() {
4     Graph G;
5     CreateGraph(G);
6
7     // Tambah node sesuai gambar pada soal
8     InsertNode(G, 'A');
9     InsertNode(G, 'B');
10    InsertNode(G, 'C');
11    InsertNode(G, 'D');
12    InsertNode(G, 'E');
13    InsertNode(G, 'F');
14    InsertNode(G, 'G');
15    InsertNode(G, 'H');
16
17    // Hubungan node sesuai gambar pada soal
18    adrNode A = FindNode(G, 'A');
19    adrNode B = FindNode(G, 'B');
20    adrNode C = FindNode(G, 'C');
21    adrNode D = FindNode(G, 'D');
22    adrNode E = FindNode(G, 'E');
23    adrNode F = FindNode(G, 'F');
24    adrNode GNode = FindNode(G, 'G');
25    adrNode H = FindNode(G, 'H');
26
27    // Koneksi sesuai gambar
28    ConnectNode(A, B);
29    ConnectNode(A, C);
30    ConnectNode(B, D);
31    ConnectNode(B, E);
32    ConnectNode(C, F);
33    ConnectNode(C, GNode);
34    ConnectNode(D, H);
35    ConnectNode(E, H);
36    ConnectNode(F, H);
37    ConnectNode(GNode, H);
38
39    // Tampilkan struktur graph
40    cout << "=== Struktur Graph ===" << endl;
41    PrintInfoGraph(G);
42
43    // Soal 2: DFS
44    cout << "=== DFS dari Node A ===" << endl;
45    ResetVisited(G);
46    PrintDFS(G, A);
47    cout << endl;
48
49    // Soal 3: BFS
50    cout << "=== BFS dari Node A ===" << endl;
51    ResetVisited(G);
52    PrintBFS(G, A);
53    cout << endl;
54
55    return 0;

```

```

#include "graph.h"

int main() {
    Graph G;
    CreateGraph(G);

    // Tambah node sesuai Gambar pada soal

```

```

InsertNode(G, 'A');
InsertNode(G, 'B');
InsertNode(G, 'C');
InsertNode(G, 'D');
InsertNode(G, 'E');
InsertNode(G, 'F');
InsertNode(G, 'G');
InsertNode(G, 'H');

// Hubungkan node sesuai Gambar pada soal
adrNode A = FindNode(G, 'A');
adrNode B = FindNode(G, 'B');
adrNode C = FindNode(G, 'C');
adrNode D = FindNode(G, 'D');
adrNode E = FindNode(G, 'E');
adrNode F = FindNode(G, 'F');
adrNode GNode = FindNode(G, 'G');
adrNode H = FindNode(G, 'H');

// Koneksi sesuai gambar
ConnectNode(A, B);
ConnectNode(A, C);
ConnectNode(B, D);
ConnectNode(B, E);
ConnectNode(C, F);
ConnectNode(C, GNode);
ConnectNode(D, H);
ConnectNode(E, H);
ConnectNode(F, H);
ConnectNode(GNode, H);

// Tampilkan struktur graph
cout << "=== Struktur Graph ===" << endl;
PrintInfoGraph(G);

// Soal 2: DFS
cout << "\n=== DFS dari Node A ===" << endl;
ResetVisited(G);
PrintDFS(G, A);
cout << endl;

// Soal 3: BFS
cout << "\n=== BFS dari Node A ===" << endl;
ResetVisited(G);
PrintBFS(G, A);
cout << endl;

return 0;

```



Screenshots Output

```
PS C:\Users\Puan_Malika\Documents\SMESTER 3\week 14> .\graphm
=== Struktur Graph ===
H -> G F E D
G -> H C
F -> H C
E -> H B
D -> H B
C -> G F A
B -> E D A
A -> C B

=== DFS dari Node A ===
A C G H F E B D

=== BFS dari Node A ===
A C B G F E D H
PS C:\Users\Puan_Malika\Documents\SMESTER 3\week 14>
```

MUHAMMAD FATHAMMUBINA
103112430188
12-IF-07

Ln 3, Col 9 45 character Plain text 100% Window UTF-8

Deskripsi:

Program di atas adalah ADT Graph (graf tak berarah) yang dibuat dengan linked list. Node berisi karakter huruf (A–H), dan setiap node menyimpan daftar edge ke tetangganya. Fungsi utama program meliputi: CreateGraph untuk menginisialisasi graf, InsertNode untuk menambah node di awal list, FindNode untuk mencari alamat node, AllocateNode/AllocateEdge untuk membuat node dan edge baru di memori, serta ConnectNode untuk menghubungkan dua node secara dua arah (karena undirected). Ada variabel visited di setiap node agar traversal tidak mengunjungi node yang sama berulang. Program mendukung DFS (rekursif) yang menelusuri graf sedalam mungkin dari node awal, dan BFS (antrian/queue) yang menelusuri graf per level. Di main, graf dibuat lalu diisi node A–H, disambungkan sesuai pola soal, kemudian strukturnya ditampilkan dengan PrintInfoGraph. Setelah itu dilakukan DFS dan BFS mulai dari node A, dengan ResetVisited dipanggil sebelum traversal agar semua node bisa dikunjungi ulang, dengan pointer agar data bisa ditambah, dicari, diurut, dan dihapus

C. Kesimpulan

Praktikum ini menunjukkan bahwa graph berhasil dibangun sebagai graf tak berarah (undirected), di mana setiap node saling terhubung secara dua arah dan dapat menampilkan daftar tetangganya dalam bentuk adjacency list. Program juga mampu melakukan traversal DFS dan BFS dari node awal setelah penanda visited di-reset, sehingga seluruh node dapat dikunjungi tanpa pengulangan. Praktikum ini membuktikan bahwa graph efektif untuk merepresentasikan hubungan antar data secara dinamis, serta memberi pemahaman bahwa proses penelusuran graph dapat disesuaikan berdasarkan kebutuhan (kedalaman atau per level). Secara keseluruhan, graph telah terbentuk sesuai koneksi yang dirancang dan traversal berjalan dengan benar.

D. Referensi

Muliono, R. (2017). Abstract Data Type (ADT). Universitas Multimedia Nusantara.

Budianto, A. P. Penerapan graf untuk struktur data himpunan saling lepas. Institut Teknologi Bandung.

Akmal, D. F., Pinandito, A., & Priyambadha, B. (2018). Rancang bangun aplikasi mobile struktur data graf berbasis Google Maps untuk rute perjalanan di lingkungan Universitas Brawijaya. Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer.